

AI CALLING AGENT

Development, Improvement & Evaluation Summary

1. Overview

The AI Calling Agent is an AI-powered outbound voice assistant for real-estate lead engagement. It is designed to handle customer enquiries, qualify leads, answer project-related FAQs and support site-visit scheduling. The solution combines telephony, speech recognition, project-knowledge retrieval, LLM response generation, speech synthesis and stage-based conversation management.

2. How the Agent Has Been Developed

- **LLM integration:** Started with local/open models such as TinyLlama and Qwen, then integrated hosted models including Sarvam AI, DeepSeek and Claude for development and comparative evaluation.
- **Project knowledge (RAG):** Project brochure content is split into text chunks; relevant information is retrieved to ground answers in approved project details.
- **Conversation flow:** Implemented stages for greeting, qualification, FAQ handling, site-visit proposal, slot confirmation and closure. Recent fixes also address context handling and site-visit sequencing.
- **Iterative application improvements:** Refined prompts, retrieval, fallback responses, response handling, state logic and configuration based on observed errors and test conversations.
- **Evaluation framework:** Built a separate, repeatable framework to compare model outputs against expected intents, actions and outcomes, while tracking factuality, context/stage handling, latency and token usage.

3. Recent Voice and STT Improvements

The live-calling setup has recently been updated using voice and speech-recognition options documented by Twilio. The aim was to improve the naturalness of the spoken voice and recognition of Indian-English responses. These are configuration/integration changes to the speech layer; they do not involve retraining the LLM.

- **Text-to-speech (voice):** Changed the configured voice from Amazon Polly standard voice `Polly.Aditi` to `Polly.Kajal-Neural`, a neural voice selected to improve naturalness for Indian-English calls.
- **Speech-to-text (STT):** The early prototype used faster-whisper. For the current live Twilio call flow, STT has moved to Twilio's native <Gather> speech-recognition configuration: model `experimental\_utterances` with language `en-IN` (Indian English). This is not Whisper or Sarvam STT. Sarvam Saaras, Google STT V2 and Deepgram were discussed as possible alternatives for a future comparison; the project notes do not establish that any of these alternatives has been adopted.
- **Conversational LLM:** The current calling flow uses Sarvam `sarvam-105b-conversations` to generate the agent's spoken responses.

- **Observed test outcome:** In the latest hands-on test, the agent's voice sounded improved and it recognised the user's responses well. These observations are an initial test, not a statistically established performance result.

4. Training vs. System Optimisation

The work to date is best described as LLM application development and evaluation-driven optimisation, rather than training a foundation model from scratch. Improvements have primarily come from model selection, prompt engineering, RAG, conversation-state logic, telephony/speech configuration and application-level changes. The available project evidence does not establish that the underlying LLM weights have been fine-tuned on a custom dataset.

5. Evaluation Progress

The evaluation report dated 23 September 2026 records 15 unique model-evaluation runs, 131 customer turns and 520 model-turn evaluations: 412 labelled and 108 unlabelled. The framework also records 149 software/framework tests; these are separate from model evaluations.

The controlled English benchmark compared Sarvam 105B Conversations, DeepSeek V4.1 Flash, Claude Sonnet 4.6 and Claude Opus 4.8 on the same customer sequence. It covered 48 evaluated model turns (12 per model), with no skipped evaluations. All four recorded 100% grounded responses and a 0% unsupported-claim rate in that benchmark. These figures apply only to the tested scenarios and do not guarantee performance across real calls.

Key metrics include intent and action accuracy, factual accuracy, grounded-response rate, unsupported-claim rate, context and stage accuracy, latency, token usage and language behaviour.